

Meridian Model and Token Strategy

Model routing, quality controls, and estimated build economics

MERIDIAN

Project record - August 26, 2026

Date: 2026-08-26

Applies to: docs/superpowers/plans/2026-08-26-meridian-overhaul-implementation.md

Recommendation

Use a quality-gated model portfolio rather than one model for the entire build:

Tier / Model / Share of agent work / Use
--- / --- / ---: / ---
Frontier / gpt-5.6-sol / 15-25% / Architecture, financial safety, migrations, ambiguous debugging, visual direction, adversarial review, release gates
Default / gpt-5.6-terra / 55-70% / TDD implementation, domain modules, APIs, responsive components, integration work, ordinary debugging
Bounded / gpt-5.6-luna / 15-25% / Repository mapping, fixture preparation, mechanical edits, test execution/summaries, documentation checks, narrow lint fixes

OpenAI's current model catalog describes Sol as the flagship for complex reasoning and coding, Terra as the intelligence/cost balance, and Luna as the cost-sensitive high-volume model. Published standard API prices are \$4/\$20, \$2/\$12, and \$0.20/\$1.20 per million input/output tokens respectively. Pricing and Codex credit accounting can differ, so dollar figures below are API-equivalent planning estimates, not a guarantee of Codex subscription consumption.

Sources:

- <https://developers.openai.com/api/docs/models>
- <https://developers.openai.com/api/docs/guides/latest-model>

Build estimate

The 26-task Meridian plan is a seven-slice rebuild touching domain modeling, migrations, provider normalization, financial invariants, AI, two responsive UI compositions, read-only evidence connectors, document extraction, and cross-media reasoning. A disciplined implementation should budget:

Slice / Input tokens / Output/reasoning tokens / Main cost drivers
--- / ---: / ---: / ---
1. Foundation and shell / 4-7M / 0.6-1.1M / Existing-system discovery, CI, concurrency safety, new shell, browser verification
2. Commitments and funding / 4-7M / 0.6-1.1M / Schema/migration reasoning, calculation tests, schedule UI
3. Providers and transaction AI / 5-9M / 0.8-1.4M / Provider edge cases, reconciliation, classification, review UX
4. Advanced intelligence / 4-7M / 0.6-1.2M / Forecast/scenarios, grounded advisor, parity audit, legacy removal
5. Document Intelligence / 4-7M / 0.7-1.2M / Encrypted evidence, email ingestion, attachment safety, extraction, reconciliation
6. Life Context / 2-4M / 0.3-0.6M / Calendar/privacy boundary, contextual assumptions, scenario integration
7. Asset and Contract Memory / 3-5M / 0.4-0.8M / Evidence-backed lifecycle model, advisory boundaries, cross-workspace UI
Total expected / 26-46M / 4.0-7.4M / Includes implementation, tool output, tests, and review rounds

At a quality-first mix of 20% Sol, 65% Terra, and 15% Luna, this is approximately **\$105-\$190 in direct API-equivalent token cost**. Allow **\$300 as the sensible ceiling** for unexpected document-parser, provider, security, and re-review work. A single-model Sol strategy could cost roughly two to three times more without proportionally improving mechanical tasks. Rework from weak unsupervised output can cost more than either strategy, so the plan does not use a bargain-only path.

Routing rules

Always use Sol

- Approval of domain and API boundaries.
- Money-movement concurrency and idempotency review.
- Database migration review against a production-data copy.
- Reconciliation logic that can double-count spending or transfers.
- Forecast invariants and financial calculation review.
- Final responsive visual review for every slice.
- Security/privacy review of AI context and provider data.
- Any debugging problem that survives two evidence-based hypotheses.

Use high reasoning normally and xhigh only for migration, concurrency, or architectural failures. Reserve max or pro mode for a release-blocking problem with explicit evaluation criteria; do not make it a default.

Use Terra by default

- Implement one plan task at a time with its tests.
- Build repositories, services, Flask Blueprints, adapters, and UI components.
- Diagnose a reproducible test or browser failure.
- Perform normal code review when the task is not financially or architecturally critical.
- Integrate reviewed pieces and update documentation.

Use medium reasoning for bounded work and high for multi-file domain or responsive-UI work.

Use Luna only for bounded work

- Locate call sites and map existing behavior.
- Generate sanitized fixtures from an explicit schema.
- Apply exact mechanical changes already specified in the plan.
- Run and summarize tests, lint, audits, and browser matrices.
- Check accessibility attributes and documented parity lists.

Luna must not independently design schemas, change money movement, decide migration mappings, invent financial calculations, or approve a release.

Quality system

Model routing is subordinate to the same gates:

- 1. Every behavioral change starts with a failing test.
- 2. Every task gets an implementation review against the spec and plan.
- 3. High-risk tasks get a separate Sol safety review.
- 4. Every slice passes unit, integration, migration, API, and browser tests.
- 5. Every slice receives a visual review at desktop and mobile sizes.
- 6. No later model is asked to infer an earlier task's unstated interface; exact interfaces live in the plan and code.

- 7. A failed review returns to the original implementer with exact evidence; it does not trigger a broad rewrite by a fresh model.
- 8. Three failed fixes to one issue stop implementation and trigger architectural review.

Context and token controls

- Give each implementation task only the approved spec, its task block, directly related files, and required interfaces.
- Do not repeatedly resend the 8,000-line app .py; extract the relevant range or move touched logic into focused modules.
- Keep stable instructions and schemas in cacheable prompt prefixes.
- Reuse persisted reasoning only while the task goal and assumptions remain stable.
- Start a fresh task context at each task boundary; carry forward interfaces, test evidence, and review findings rather than the full transcript.
- Run tool-heavy searches and test-log reduction programmatically so raw logs do not repeatedly enter model context.
- Track tokens by task, model, test retries, and review rounds. Investigate any task exceeding twice its estimate before continuing.

Official OpenAI guidance reports that leaner prompts and tool descriptions improved internal coding-agent evaluations while reducing token use substantially; this should be validated against Meridian tasks rather than accepted blindly.

OpenRouter and Ox Alpha

Do not make the opaque openrouter/stealth/ox-alpha alias the primary development or production intelligence layer. An alias whose underlying model can change weakens reproducibility, evaluation, and incident diagnosis.

OpenRouter can remain:

- a rate-limit or availability fallback;
- an optional second opinion on a non-mutating design question;
- a development experiment behind the same structured-output and evaluation harness.

Production AI should use a pinned, named model and record provider, model id, prompt version, latency, and usage with every classification/advisor result. Provider failover must not silently change schema behavior or confidence calibration.

Runtime AI cost after launch

Build cost and app-operating cost are separate. Transaction classification should be batched and use deterministic rules before AI. At personal-finance volume, a Luna-class model should make classification inexpensive-typically cents to low single-digit dollars per month depending on history included, retries, and explanation length. The contextual advisor can use Terra for ordinary questions and escalate to Sol only for complex scenario comparison.

Recommended runtime route:

- Deterministic rules first: no model call.
- Transaction classification: Luna, structured output, small batches, cached taxonomy.
- Ordinary advisor and explanations: Terra at low/medium reasoning.
- Complex scenario narrative: Terra high; Sol only on explicit high-value request.
- All arithmetic, forecasts, and money proposals: deterministic Meridian services; models explain results rather than calculate authoritative balances.

Budget controls

- Set a per-task soft budget and a per-slice hard review threshold.
- Alert at 50%, 75%, and 90% of the monthly API budget.
- Disable automatic Sol escalation after the ceiling; require owner approval.
- Cache stable taxonomy, schema, and product rules.
- Store token usage returned by providers and display monthly totals under Connections.
- Maintain an evaluation set covering classification, reconciliation explanation, schedule advice, and refusal to mutate without approval.

The recommended strategy is therefore **Terra-led implementation, Sol-governed quality, and Luna-assisted throughput**. It preserves the highest standard at every irreversible or judgment-heavy boundary while avoiding premium-token waste on deterministic work.